Universidad de Montemorelos

Facultad de Ingeniería y Tecnología

Materia: Estándares Web

Nombre del proyecto: TaskFlow

Nombre del maestro: Daniel Neri Ramírez

Nombre del alumno: Hiday Misael Servin Ruiz

Fecha: 01 de Abril del 2026

GitHub: https://github.com/Misa-ISC/Robotsito-con-Llantitas#

1. server.js — El servidor

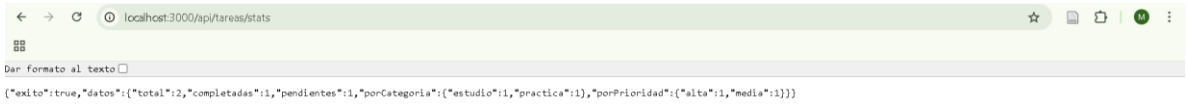
Este archivo es el backend de la app. Corre en Node.js y levanta un servidor en el puerto 3000 usando Express. Tiene tres cosas principales: CORS para que el navegador no bloquee las peticiones, express.json() para leer los datos que llegan del frontend, y express.static() para servir los archivos HTML, CSS y JS al navegador.

En lugar de una base de datos real, las tareas se guardan en un array en memoria. Funciona bien para la práctica aunque los datos se borran al reiniciar el servidor.

```
index.html   styles.css   app.js   server.js   package.json
taskflow > server.js > ...
1  // server.js
2  const express = require("express");
3  const cors = require("cors");
4
5  const app = express();
6  const PORT = 3000;
7
8  // Middleware
9  app.use(cors());
10 app.use(express.json());
11 app.use(express.static("public"));
12
13 // Base de datos en memoria (array de tareas)
14 let tareas = [
15   {
16     id: 1,
17     titulo: "Aprender HTML5 semántico",
18     descripcion: "Estudiar las etiquetas header, nav, main, section, article y footer",
19     categoria: "estudio",
20     prioridad: "alta",
21     completada: false,
22     fechaCreacion: new Date().toISOString()
23   },
24   {
25     id: 2,
26     titulo: "Practicar CSS Grid",
27     descripcion: "Crear layouts con grid-template-columns y grid-template-rows",
28     categoria: "practica",
29     prioridad: "media",

PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   PUERTOS
node

[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node ../server.js`
Servidor TaskFlow corriendo en http://localhost:3000

Lín. 156, col. 4   Espacios: 2   UTF-8   CRLF   {} JavaScript   Go Live
```

2. Las rutas de la API

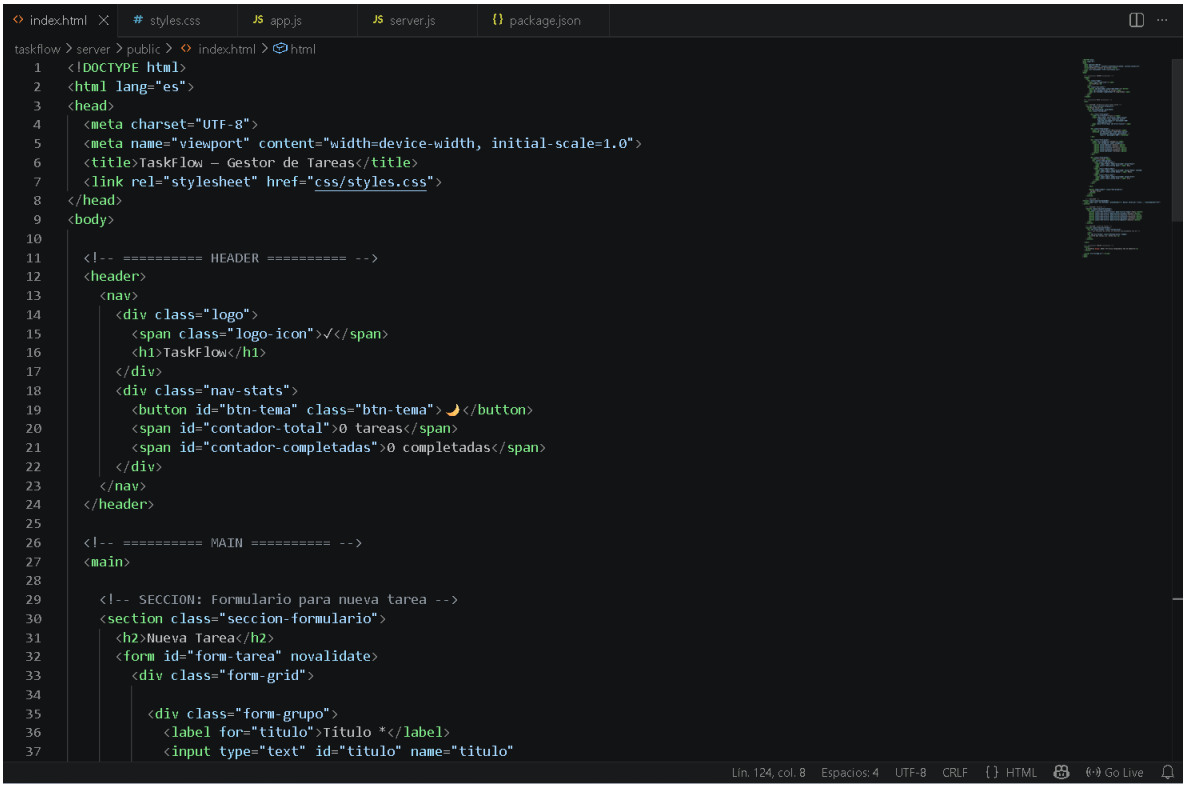
La API tiene 7 rutas que cubren todo lo necesario. GET /api/tareas devuelve todas las tareas y permite filtrar por categoría. GET /api/tareas/stats devuelve estadísticas como total, completadas y pendientes. GET /api/tareas/:id regresa una tarea específica. POST crea una nueva tarea. PUT actualiza sus datos. PATCH cambia el estado de completada a pendiente o al revés. DELETE la elimina.



---

3. index.html — La estructura

Usa etiquetas semánticas de HTML5 en lugar de puro div. El header tiene el logo y los contadores, el main tiene el formulario, los filtros y la lista de tarjetas, y el footer los créditos. Cada tarjeta se genera como un article. Los atributos data-id y data-filtro conectan el HTML con JavaScript sin necesidad de IDs extra.




```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TaskFlow — Gestor de Tareas</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>

  <!-- ========== HEADER ========== -->
  <header>
    <nav>
      <div class="logo">
        <span class="logo-icon">✓</span>
        <h1>TaskFlow</h1>
      </div>
      <div class="nav-stats">
        <button id="btn-tema" class="btn-tema">🌙</button>
        <span id="contador-total">0 tareas</span>
        <span id="contador-completadas">0 completadas</span>
      </div>
    </nav>
  </header>

  <!-- ========== MAIN ========== -->
  <main>

    <!-- SECCION: Formulario para nueva tarea -->
    <section class="seccion-formulario">
      <h2>Nueva Tarea</h2>
      <form id="form-tarea" novalidate>
        <div class="form-grid">

          <div class="form-grupo">
            <label for="titulo">Título *</label>
            <input type="text" id="titulo" name="titulo"
```

4. styles.css — Los estilos

Al inicio se definen variables CSS con los colores y transiciones del tema. Esto hace que el modo oscuro sea fácil: solo sobreescribe esas variables en body.oscuro. El layout usa Flexbox en el header y los filtros, y CSS Grid en el formulario y las tarjetas. Hay animaciones de entrada y salida para las tarjetas, y media queries que adaptan todo a pantallas pequeñas.

```
index.html   styles.css   app.js   server.js   package.json
taskflow > server > public > css > styles.css > .tarea-card.eliminando
/* ===== VARIABLES CSS (Custom Properties) ===== */
:root {
  --color-primario: #2563eb;
  --color-primario-hover: #1d4ed8;
  --color-fondo: #f0f4f8;
  --color-superficie: #ffffff;
  --color-texto: #1e293b;
  --color-texto-secundario: #64748b;
  --color-borde: #e2e8f0;
  --color-exito: #16a34a;
  --color-peligro: #dc2626;
  --color-advertencia: #f59e0b;
  --prioridad-alta: #ef4444;
  --prioridad-media: #f59e0b;
  --prioridad-baja: #22c55e;
  --sombra: 0 1px 3px rgba(0, 0, 0, 0.1);
  --sombra-hover: 0 4px 12px rgba(0, 0, 0, 0.15);
  --radio-borde: 8px;
  --transicion: 0.3s ease;
}

/* ===== RESET Y BASE ===== */
*, *::before, *::after {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: "Segoe UI", system-ui, -apple-system, sans-serif;
  background-color: var(--color-fondo);
  color: var(--color-texto);
  line-height: 1.6;
  min-height: 100vh;
  display: flex;
  flex-direction: column;
}
Lín. 476, col. 2   Espacios: 4   UTF-8   CRLF   {} CSS   Go Live
```

5. app.js — La lógica del cliente

Es el JavaScript del navegador. Usa Fetch API con async/await para hablar con el servidor. Hay una función para cada operación: obtener, crear, toggle y eliminar tareas. Las tarjetas se generan con template literals y se insertan directo en el DOM. Los filtros usan delegación de eventos: un solo listener en el contenedor detecta qué botón se clickeó.

index.html | styles.css | app.js | server.js | package.json

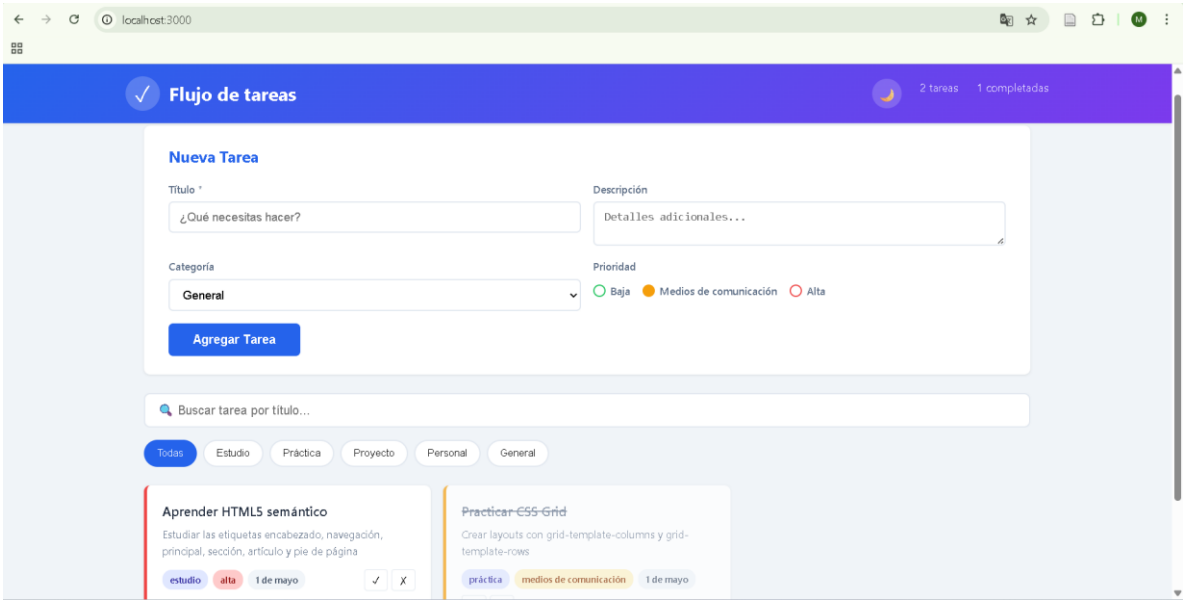
taskflow > server > public > js > app.js > manejarEliminar

```javascript
// ===== app.js - Lógica del cliente TaskFlow =====

// URL base de la API (nuestro servidor Express)
const API_URL = "/api/tareas";

// Referencias al DOM usando querySelector
const formTarea = document.querySelector("#form-tarea");
const listaTareas = document.querySelector("#lista-tareas");
const sinTareas = document.querySelector("#sin-tareas");
const contadorTotal = document.querySelector("#contador-total");
const contadorCompletadas = document.querySelector("#contador-completadas");
const filtrosContenedor = document.querySelector(".filtros-contenedor");
const errorTitulo = document.querySelector("#error-titulo");

// Estado de la aplicación
let filtroActual = "todas";
let busquedaActual = "";

// ===== FUNCIONES PARA CONSUMIR LA API (Fetch + async/await) =====

/**
 * Obtiene las tareas del servidor.
 * Usa Fetch API con async/await.
 */
async function obtenerTareas(categoria = "todas") {
  try {
    const queryParam = categoria !== "todas" ? `?categoria=${categoria}` : "";
    const respuesta = await fetch(`${API_URL}${queryParam}`);

    if (!respuesta.ok) {
      throw new Error(`Error HTTP: ${respuesta.status}`);
    }

    const datos = await respuesta.json();
    return datos.datos;
  } catch (error) {
    console.error("Error al obtener tareas:", error);
```

Lín. 244, col. 2 Espacios: 4 UTF-8 CRLF { } JavaScript Go Live

6. Extras implementados

Se hicieron los 4 retos adicionales. Búsqueda en tiempo real con Array.filter() sobre los títulos. Modo oscuro con classList.toggle y localStorage para guardar la preferencia. Animación de salida al eliminar con CSS keyframes y setTimeout. Endpoint de estadísticas en el servidor usando Array.reduce() para agrupar por categoría y prioridad.


localhost:3000
Flujo de tareas
2 tareas
1 completadas
Nueva Tarea
Título *
¿Qué necesitas hacer?
Descripción
Detalles adicionales...
Categoría
General
Prioridad
Baja
Medios de comunicación
Alta
Agregar Tarea
Buscar tarea por título...
Todas
Estudio
Práctica
Proyecto
Personal
General
Aprender HTML5 semántico
Estudiar las etiquetas encabezado, navegación, principal, sección, artículo y pie de página
estudio
alta
1 de mayo
Practicar CSS Grid
Crear layouts con grid-template-columns y grid-template-rows
práctica
medios de comunicación
1 de mayo